

Actividad Práctica - Sesión 8

Integración del módulo Productos con Categorías

Dato	Detalle	Dato	Detalle
Unidad	II. Frontend SPA empresarial seguro	Sesión	8 · 1 de octubre de 2026
Modalidad	Individual, en laboratorio	Duración	2 horas prácticas
Tecnologías	Angular 22 · TypeScript · Spring Boot 4	Proyecto base	PharmaSoft (sesión 7) + PharmaBackend
Docente	Reyna Barreto Benjamin David	Ciclo	IV · Semestre 2026-2
Estudiante	Kevin Eduardo Del Carpio Alegría	Proyecto	PharmaSoft

Se documentan la implementación y la verificación de doce escenarios del módulo Productos: listado, registro, edición, selección de categoría y baja lógica.

B1. Resumen de la ejecución

Se verificaron doce escenarios en la SPA de localhost:4200 y la API de localhost:8080/api/v1, con datos QA. En la primera ejecución, once escenarios resultaron satisfactorios y uno falló. Tras corregir la eliminación de categorías referenciadas, la repetición del caso 12 resultó satisfactoria. Se mantuvieron separadas las evidencias históricas y las posteriores a la corrección.

Grupo de escenarios	Casos	Pasan inicialmente	Fallan inicialmente
Listado, orden, paginación y filtro	4	4	0
Alta y validación del formulario	4	4	0
Edición y cambio de categoría	2	2	0
Baja y eliminación de categoría	2	1	1
Total	12	11	1

La clasificación corresponde a la práctica. CategoriaDeletionTest (3 pruebas) y ProductoContractTest (4 pruebas) finalizaron sin fallos. El módulo se integró en la rama "develop". Las verificaciones complementarias de Codex están identificadas en sus registros.

B1. Verificación del escenario P01

01. Listado inicial

GET /productos con pagina=0, tamaño=10, ordenarPor=nombre y direccion=asc: 200. La tabla muestra categorías y Productos está seleccionado en el menú.

The screenshot shows the PharmaSoft application interface. On the left, there's a sidebar with 'MÓDULOS' including 'Inicio', 'Categorías', 'Clientes', and 'Productos'. The main area displays the 'Productos' page with a filter for 'Todas' and 'Productos por página' set to 10. A table lists products with columns: Nombre, Categoría, Precio, Stock, Estado, and Acciones. The table shows products like 'Aceite de Oliva', 'Aceite de Palma', 'Laptop ThinkPad', 'Leche Gloria', and 'Leche Laive'. On the right, the DevTools Network tab shows a GET request to 'http://localhost:8080/api/v1/productos?pagina=0&tamano=10&ordenarPor=nombre&direccion=asc' with a status of 200 OK.

B1. Verificación del escenario P02

02. Ordenar precio: ascendente

GET con ordenarPor=precio y direccion=asc: 200. Los precios visibles aumentan desde S/ 0.20.

The screenshot shows the PharmaSoft application interface. The 'Productos' page now displays products sorted by price in ascending order. The table shows products like 'Aceite de Palma' (S/ 0.20), 'QA S8 DelCarpio Producto 01' (S/ 1.50), 'QA S8 DelCarpio Producto 02' (S/ 2.50), and 'QA S8 DelCarpio Producto 03' (S/ 3.50). The DevTools Network tab shows a GET request to 'http://localhost:8080/api/v1/productos?pagina=0&tamano=10&ordenarPor=precio&direccion=asc' with a status of 200 OK.

02. Ordenar precio: descendente

GET con ordenarPor=precio y direccion=desc: 200. El mayor precio visible encabeza el listado.

The screenshot shows the PharmaSoft application interface. The 'Productos' page now displays products sorted by price in descending order. The table shows products like 'QA Ibuprofeno' (S/ 20.00), 'QA Cuidado Personal' (S/ 15.00), 'QA Paracetamol' (S/ 10.00), and 'Laptop ThinkPad' (S/ 1,200.50). The DevTools Network tab shows a GET request to 'http://localhost:8080/api/v1/productos?pagina=0&tamano=10&ordenarPor=precio&direccion=desc' with a status of 200 OK.

B1. Verificación del escenario P03

03. Tamaño de página y navegación

Tras elegir tamaño 5 y pulsar Siguiente, GET con pagina=1 y tamaño=5: 200. El índice enviado es contado desde cero.

The screenshot shows the PharmaSoft application interface. On the left, there's a sidebar with 'MÓDULOS' including 'Inicio', 'Categorías', 'Clientes', and 'Productos'. The main area is titled 'Productos' and features a filter for 'Categoría' (set to 'Todas') and 'Productos por página' (set to '5'). Below this, a table lists products with columns: Nombre, Categoría, Precio, Stock, Estado, and Acciones. The table contains four rows of product data. To the right, the DevTools network log shows a GET request to 'http://localhost:3000/api/v1/productos/pagina=1&tamaño=5&ordenarPor=precio&direccion=desc' with a status of 200 OK.

03. Complemento: paginador

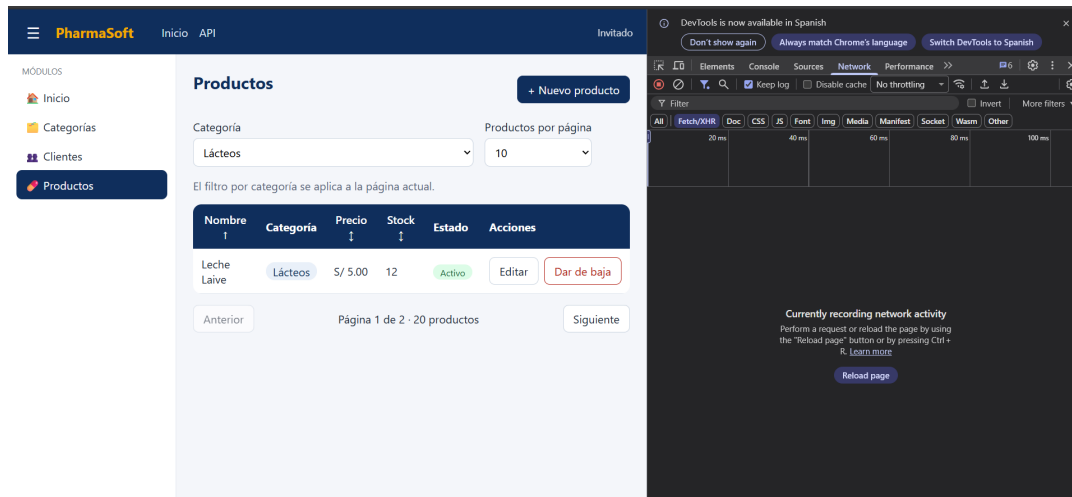
Captura anterior de la misma funcionalidad: tamaño 5 y Página 2 de 4. El total depende de los datos existentes en el momento de cada ejecución.

This screenshot shows the PharmaSoft application with the 'Productos' page. The 'Categoría' filter is still 'Todas' and 'Productos por página' is '5'. The table now displays five rows of product data. At the bottom of the table, there is a pagination control showing 'Página 2 de 4'.

B1. Verificación del escenario P04

04. Filtro por categoría

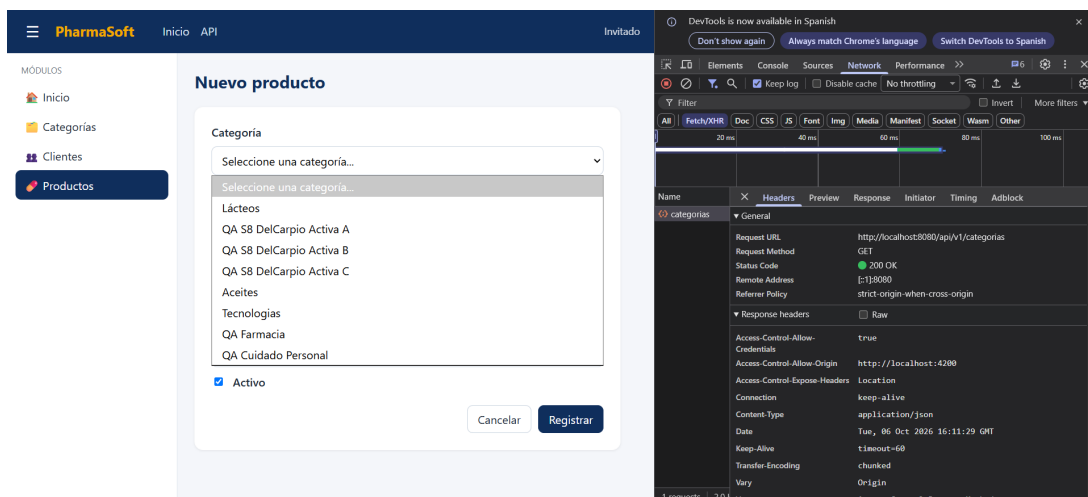
Categoría Lácteos: la página muestra solo su producto. Network vacío después de limpiar y cambiar el filtro: el filtrado se aplica localmente a la página actual.



B1. Verificación del escenario P05

05. Categorías activas en el formulario

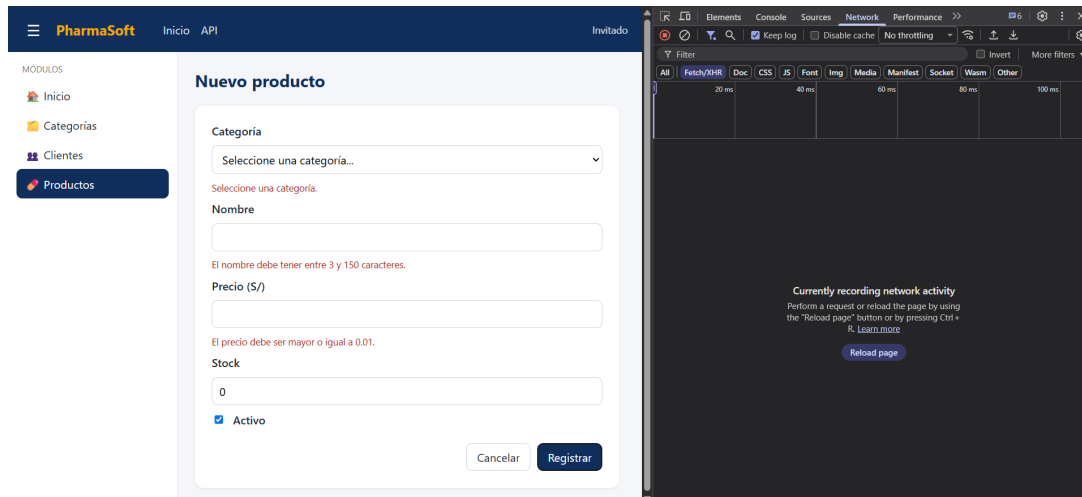
GET /categorias: 200. El select desplegado ofrece categorías activas; QA S8 DelCarpio Inactiva (49) no aparece para un registro nuevo.



B1. Verificación del escenario P06

06. Validación sin petición HTTP

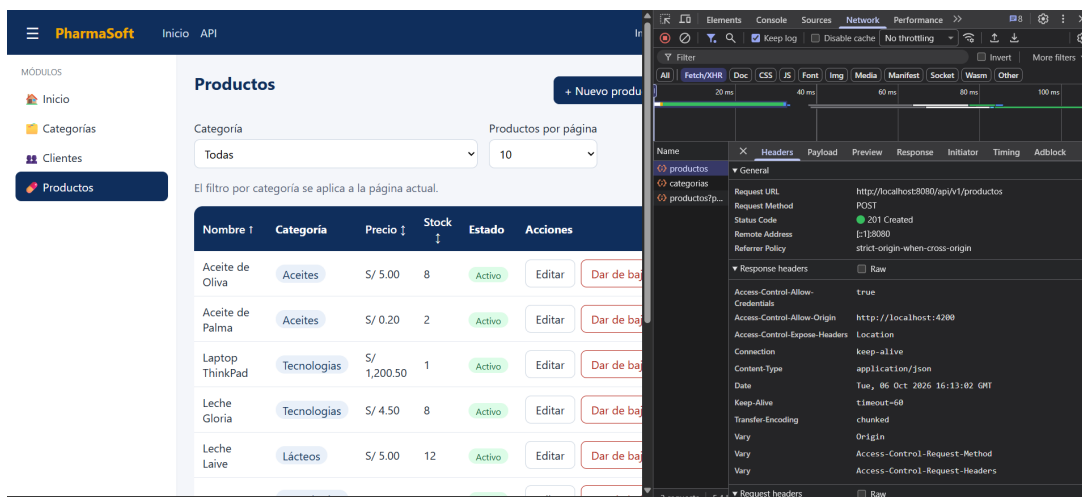
Registrar un formulario vacío muestra errores de categoría, nombre y precio. Network permanece vacío: la SPA bloquea el envío.



B1. Verificación del escenario P07

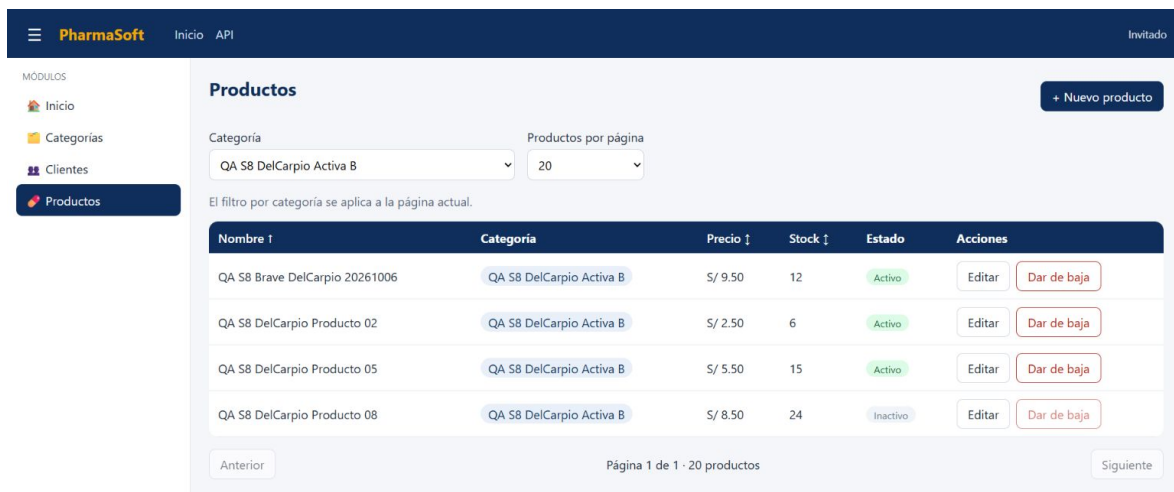
07. Registro válido: respuesta HTTP

POST /productos: 201 Created. Esta captura demuestra la respuesta de creación; la fila nueva queda fuera del área visible.



07. Complemento: registro en el listado

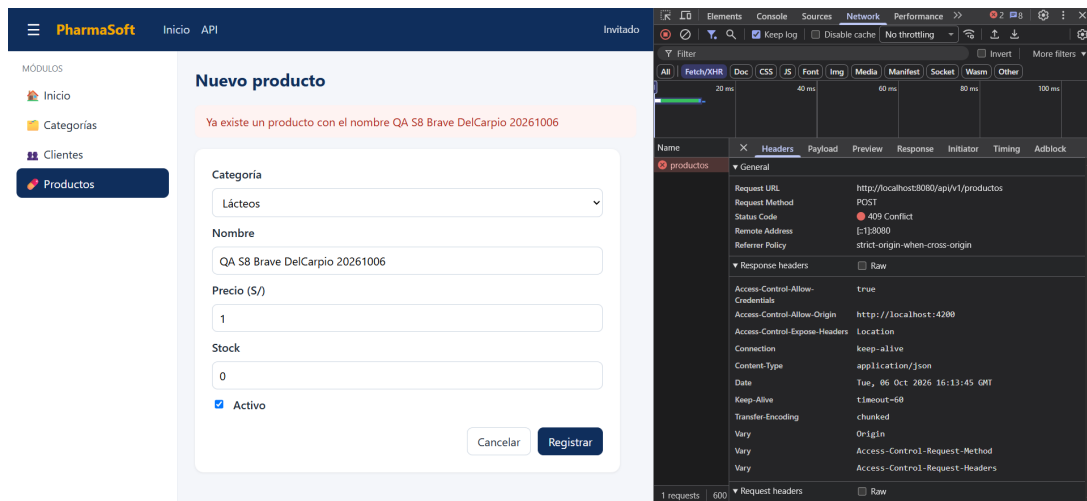
Producto creado durante la ejecución previa: QA S8 Brave DelCarpio 20261006 aparece en el listado con categoría activa, tras su edición. Confirma la presencia del registro; no se atribuye a la misma petición Network documentada.



B1. Verificación del escenario P08

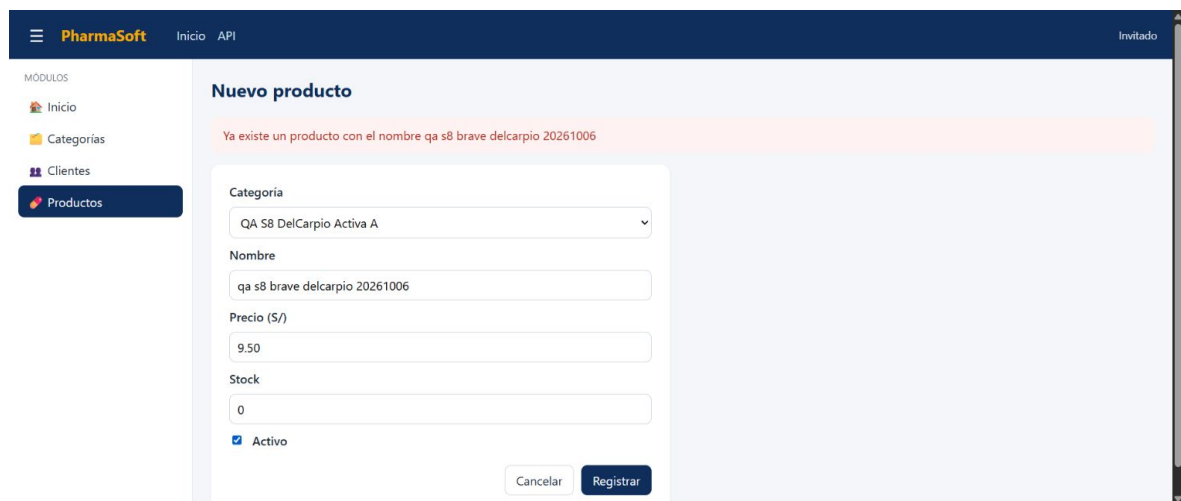
08. Nombre duplicado: conflicto HTTP

POST /productos: 409 Conflict y mensaje de duplicado. Esta captura usa la capitalización original; la siguiente documenta el intento con minúsculas solicitado por la guía.



08. Complemento: nombre en minúsculas

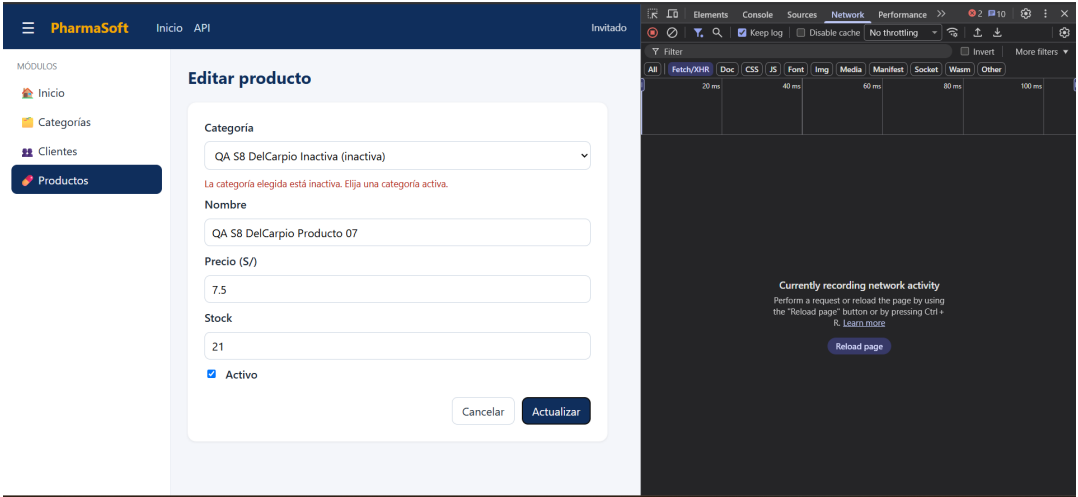
La ejecución anterior intentó registrar el nombre existente en minúsculas y mostró el mensaje de duplicado. El control compara nombres sin distinguir mayúsculas y minúsculas.



B1. Verificación del escenario P09

09. Categoría original inactiva

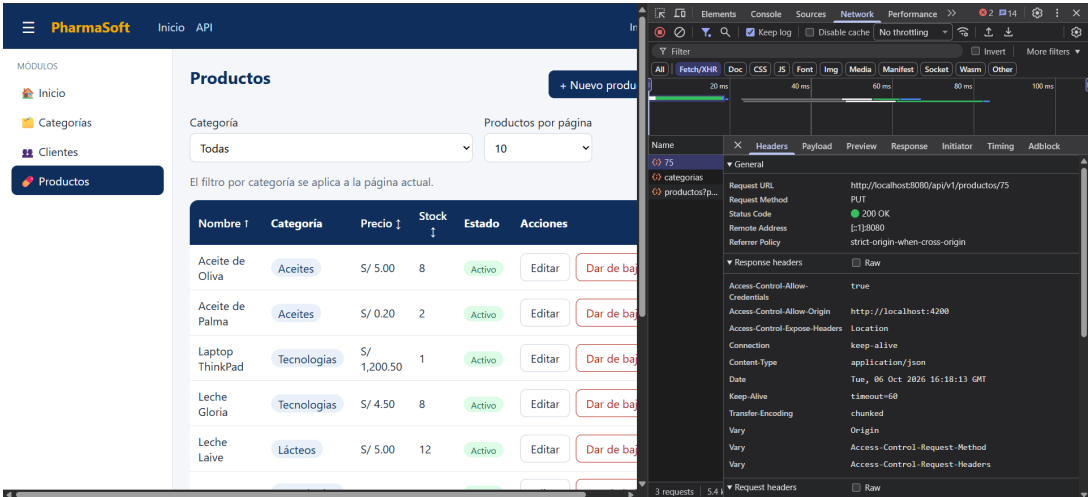
Al editar el producto 73, el select conserva la categoría original con (inactiva). El aviso explica que debe elegirse una activa; Actualizar no envía una petición.



B1. Verificación del escenario P10

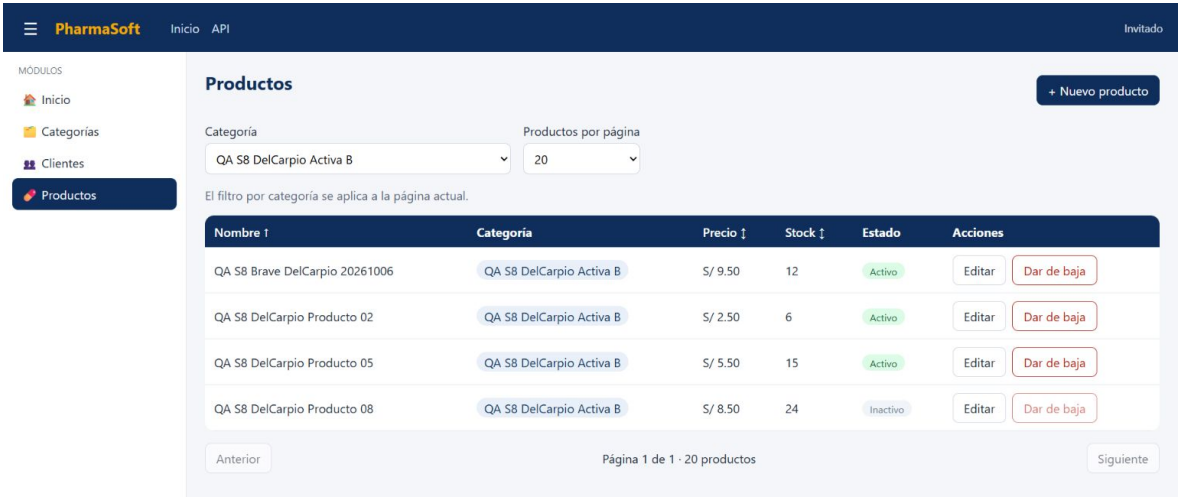
10. Cambio de categoría: respuesta HTTP

PUT /productos/75: 200 OK. La fila actualizada queda fuera del área visible de esta captura; el complemento muestra una reasignación previa del mismo producto.



10. Complemento: categoría actualizada

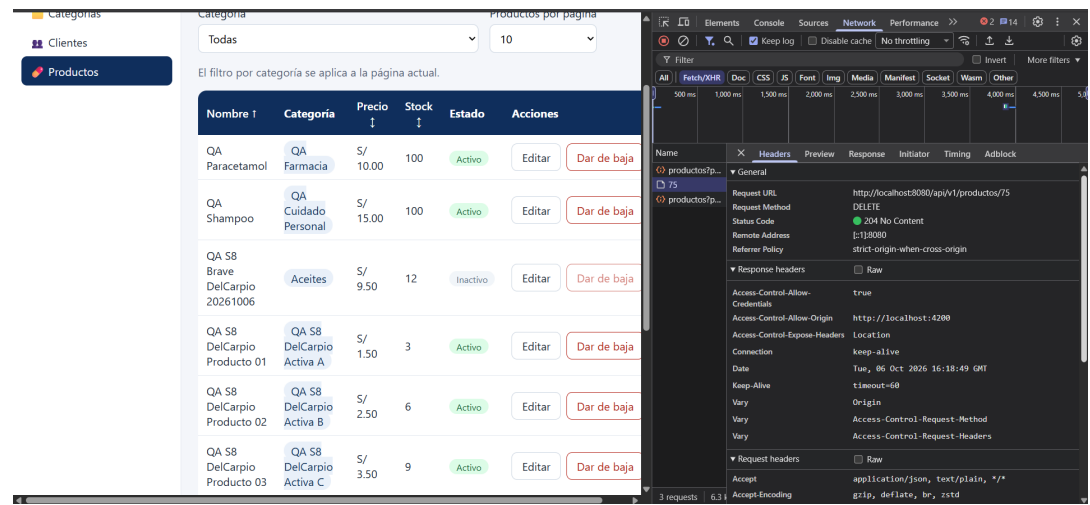
Ejecución anterior: producto 75 reasignado de categoría 46 a 47. En una edición posterior documentada se cambió a Aceites; esta categoría se observa en el caso 11.



B1. Verificación del escenario P11

11. Baja lógica del producto

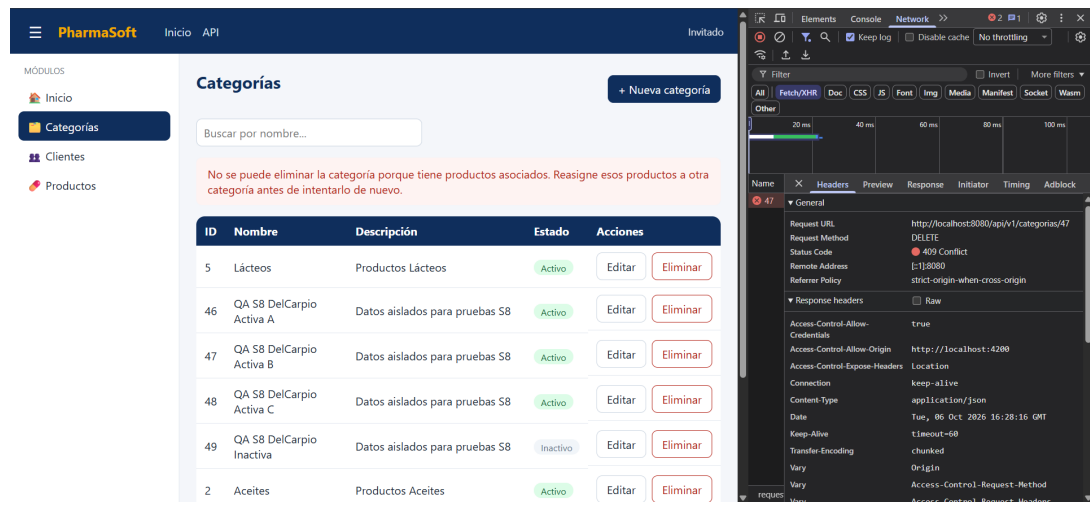
DELETE /productos/75: 204 No Content. La fila permanece, muestra Inactivo y el botón Dar de baja está deshabilitado.



B1. Verificación del escenario P12

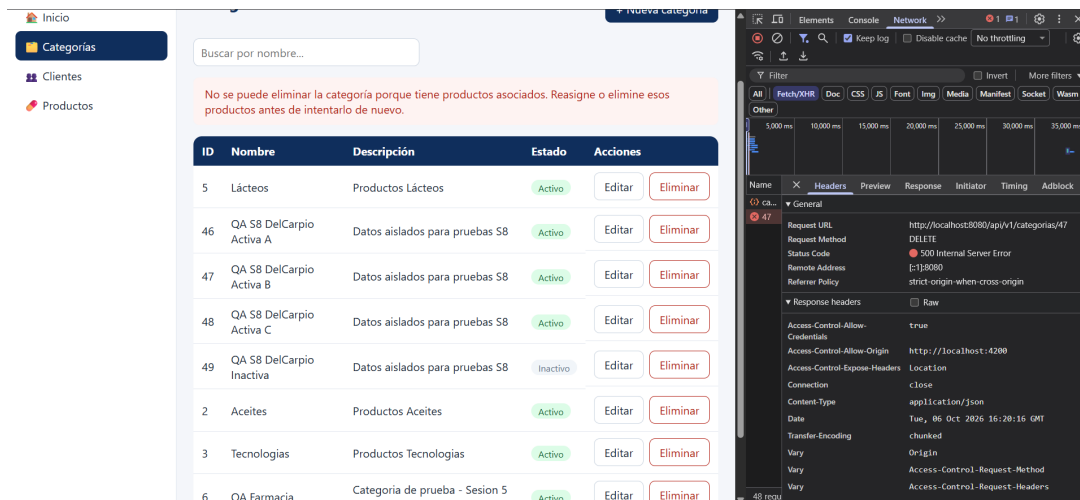
12. Categoría referenciada: repetición correcta

DELETE /categorias/47: 409 Conflict. El mensaje explica la dependencia y pide reasignar productos. La categoría 47 permanece en la tabla. Resultado final del caso: pasa.



12. Resultado histórico antes de la corrección

Evidencia histórica del defecto: DELETE /categorias/47 devolvía 500 aunque la SPA mostraba un mensaje claro. Este resultado fue reemplazado por la repetición con respuesta 409; no representa el comportamiento final.



B2. Ficha del hallazgo P12

ID: H-P01. **Severidad:** media. La eliminación de categoría referenciada devolvió 500 en vez de un conflicto de negocio. `CategoriaServiceImpl.delete()` intentaba borrar antes de comprobar las referencias. Se incorporó `ProductoRepository.existsByCategoryId` y `ReglaNegocioException`; `GlobalExceptionHandler` devuelve 409. La repetición conservó la categoría, incluidos los vínculos de productos inactivos. Las dos capturas anteriores documentan el fallo y la corrección.

B3. Cobertura de reglas

La tabla describe la cobertura observada durante la verificación del módulo Productos.

Regla	Cobertura en la práctica	Referencia y comportamiento
Existencia de categoría	Ambas	el formulario de productos y <code>ProductoServiceImpl</code> : selección y comprobación de referencia
Categoría activa	Solo SPA	el formulario de productos: ofrece activas y bloquea la categoría original inactiva
Nombre único	Solo API en alta	<code>ProductoServiceImpl.create()</code> : conflicto por nombre; la SPA muestra el error recibido
No eliminar categoría con productos	Solo API	<code>CategoriaServiceImpl.delete()</code> : consulta de referencias y rechazo 409 tras la corrección
Baja lógica	Ambas	<code>ProductoServiceImpl</code> cambia estado; el listado de productos impide repetir desde la fila inactiva

Mostrar un error devuelto por el servidor no constituye una validación independiente de la SPA. La prueba del nombre duplicado verifica el rechazo de la API y la presentación del mensaje en el formulario.

B4. Recomendación sobre el filtro por categoría

`ProductoList` filtra `resultado().contenido` después de recibir una página. Por ello, la selección oculta registros de esa página y no recupera productos de páginas distintas. Los totales siguen describiendo la consulta global; aumentar el tamaño a 100 no convierte el filtro en una consulta global.

Endpoint: agregar `categoryId` opcional a `GET /api/v1/productos` y conservar `pagina`, `tamano`, `ordenarPor` y `direccion`.

Repositorio: incorporar `Page<Producto> findByCategoryId(Long categoryId, Pageable pageable)` en `ProductoRepository`. El servicio debe usar la consulta filtrada cuando exista `categoryId` y `findAll(pageable)` cuando no exista; el total debe corresponder a la consulta elegida.

Frontend: `ProductoService.listar()` debe enviar `categoryId` como parámetro opcional. `ProductoList` debe reiniciar `pagina` a 0 al cambiar la categoría, volver a consultar y eliminar el filtrado local. El total y la navegación deben representar

únicamente los productos de la categoría seleccionada. La consulta global por categoría sigue siendo una propuesta de mejora.

B5. Preguntas de análisis

El análisis aborda la protección de las referencias entre Productos y Categorías y las responsabilidades de la SPA y la API.

1. ¿Por qué no basta con ocultar categorías inactivas? La lista de el formulario de productos orienta la selección, pero una petición directa puede enviar cualquier ID. La ocultación local no impide enviar una referencia inactiva directamente a la API. `ProductoServiceImpl.create()` y `update()`, deben comprobar tanto existencia como estado al guardar.

2. ¿Qué debe ocurrir al desactivar una categoría con productos activos? La desactivación debe rechazarse mientras existan productos activos, para conservar una clasificación válida de los artículos disponibles para venta. `CategoriaServiceImpl.update()` debe consultar `ProductoRepository.existsByCategorialdAndEstadoTrue`. La reasignación o baja debe ser una decisión explícita, sin desactivar productos automáticamente.

3. ¿Deben los productos dados de baja impedir eliminar la categoría? Debe conservarse la referencia, porque la baja lógica mantiene el artículo y su clasificación histórica. Eliminar la categoría podría perder el contexto de los reportes de ventas. `CategoriaServiceImpl.delete()` y `ProductoRepository.existsByCategoriald` deben considerar cualquier producto vinculado, incluido el inactivo.

4. ¿Cuándo deben validarse las dependencias? Una categoría válida al abrir el formulario puede quedar inactiva antes del envío. Solo el backend puede consultar y proteger el estado actual al guardar. `ProductoServiceImpl.create()` debe revalidarlo dentro de la transacción y coordinar su bloqueo con `CategoriaServiceImpl.update()`; la lista local de el formulario de productos no garantiza ese estado.

5. ¿Qué cambia en la SPA al reforzar la validación de categorías activas? El filtro de categorías activas de el formulario de productos debe mantenerse. El manejo mediante `mensajeError` y el `signal error` permite mostrar el nuevo 409 sin perder los datos ingresados. Deben actualizarse los resultados esperados y verificarse el rechazo de un formulario obsoleto; la prevención local y la presentación de errores siguen siendo necesarias.